

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Country': [ 'France', 'Spain', 'Germany', 'Spain', 'Germany', 'France', 'Spain', 'France', np.nan, 'France' ],
    'Age': [ 44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37 ],
    'Salary': [ 72000, 48000, 54000, 61000, np.nan, 58000, 52000, 79000, 83000, 67000 ],
    'Purchased': [ 'No', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes' ]
})

df
```

	Country	Age	Salary	Purchased	
0	France	44.0	72000.0	No	
1	Spain	27.0	48000.0	Yes	
2	Germany	30.0	54000.0	No	
3	Spain	38.0	61000.0	No	
4	Germany	40.0	NaN	Yes	
5	France	35.0	58000.0	Yes	
6	Spain	NaN	52000.0	No	
7	France	48.0	79000.0	Yes	
8	NaN	50.0	83000.0	No	
9	France	37.0	67000.0	Yes	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.Country.fillna(df.Country.mode()[0], inplace=True)
df
```

/tmp/ipykernel_2465/1453460914.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c...
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are...
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df.Country.fillna(df.Country.mode()[0], inplace=True)
```

	Country	Age	Salary	Purchased	
0	France	44.0	72000.0	No	
1	Spain	27.0	48000.0	Yes	
2	Germany	30.0	54000.0	No	
3	Spain	38.0	61000.0	No	
4	Germany	40.0	NaN	Yes	
5	France	35.0	58000.0	Yes	
6	Spain	NaN	52000.0	No	
7	France	48.0	79000.0	Yes	
8	France	50.0	83000.0	No	
9	France	37.0	67000.0	Yes	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
features
```

```
array([[ 'France', 44.0, 72000.0],
       [ 'Spain', 27.0, 48000.0],
       [ 'Germany', 30.0, 54000.0],
       [ 'Spain', 38.0, 61000.0],
       [ 'Germany', 40.0, nan],
       [ 'France', 35.0, 58000.0],
       [ 'Spain', nan, 52000.0],
       [ 'France', 48.0, 79000.0],
       [ nan, 50.0, 83000.0],
       [ 'France', 37.0, 67000.0]], dtype=object)
```

```

from sklearn.impute import SimpleImputer

age = SimpleImputer(strategy="mean", missing_values=np.nan)
Salary = SimpleImputer(strategy="mean", missing_values=np.nan)

age.fit(features[:, [1]])
Salary.fit(features[:, [2]])

features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = Salary.transform(features[:, [2]])

features

```

```

array([[ 'France', 44.0, 72000.0],
       [ 'Spain', 27.0, 48000.0],
       [ 'Germany', 30.0, 54000.0],
       [ 'Spain', 38.0, 61000.0],
       [ 'Germany', 40.0, 63777.77777777778],
       [ 'France', 35.0, 58000.0],
       [ 'Spain', 38.77777777777778, 52000.0],
       [ 'France', 48.0, 79000.0],
       [nan, 50.0, 83000.0],
       [ 'France', 37.0, 67000.0]], dtype=object)

```

```

from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])
Country

```

```

array([[1., 0., 0., 0.],
       [0., 0., 1., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 1., 0., 0.],
       [1., 0., 0., 0.],
       [0., 0., 1., 0.],
       [1., 0., 0., 0.],
       [0., 0., 0., 1.],
       [1., 0., 0., 0.]])

```

```

final_set = np.concatenate((Country, features[:, [1,2]]), axis=1)
final_set

```

```

array([[1.0, 0.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 0.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 0.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 0.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 0.0, 48.0, 79000.0],
       [0.0, 0.0, 0.0, 1.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)

```

```

from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc.fit(final_set)

feat_standard_scaler = sc.transform(final_set)
feat_standard_scaler

```

```

array([[ 1.22474487e+00, -5.00000000e-01, -6.54653671e-01,
        -3.33333333e-01,  7.58874362e-01,  7.49473254e-01],
       [-8.16496581e-01, -5.00000000e-01,  1.52752523e+00,
        -3.33333333e-01, -1.71150388e+00, -1.43817841e+00],
       [-8.16496581e-01,  2.00000000e+00, -6.54653671e-01,
        -3.33333333e-01, -1.27555478e+00, -8.91265492e-01],
       [-8.16496581e-01, -5.00000000e-01,  1.52752523e+00,
        -3.33333333e-01, -1.13023841e-01, -2.53200424e-01],
       [-8.16496581e-01,  2.00000000e+00, -6.54653671e-01,
        -3.33333333e-01,  1.77608893e-01,  6.63219199e-16],
       [ 1.22474487e+00, -5.00000000e-01, -6.54653671e-01,
        -3.33333333e-01, -5.48972942e-01, -5.26656882e-01],
       [-8.16496581e-01, -5.00000000e-01,  1.52752523e+00,
        -3.33333333e-01,  0.00000000e+00, -1.07356980e+00],
       [ 1.22474487e+00, -5.00000000e-01, -6.54653671e-01,
        -3.33333333e-01,  1.34013983e+00,  1.38753832e+00],
       [-8.16496581e-01, -5.00000000e-01, -6.54653671e-01,
        3.00000000e+00,  1.63077256e+00,  1.75214693e+00],
       [ 1.22474487e+00, -5.00000000e-01, -6.54653671e-01,
        -3.33333333e-01, -2.58340208e-01,  2.93712492e-01]])

```

```

from sklearn.preprocessing import MinMaxScaler

```

```
mms = MinMaxScaler(feature_range=(0,1))  
mms.fit(final_set)  
  
feat_minmax_scaler = mms.transform(final_set)  
feat_minmax_scaler
```

```
array([[1.        , 0.        , 0.        , 0.        , 0.73913043,  
       0.68571429],  
       [0.        , 0.        , 1.        , 0.        , 0.        ,  
        0.        ],  
       [0.        , 1.        , 0.        , 0.        , 0.13043478,  
        0.17142857],  
       [0.        , 0.        , 1.        , 0.        , 0.47826087,  
        0.37142857],  
       [0.        , 1.        , 0.        , 0.        , 0.56521739,  
        0.45079365],  
       [1.        , 0.        , 0.        , 0.        , 0.34782609,  
        0.28571429],  
       [0.        , 0.        , 1.        , 0.        , 0.51207729,  
        0.11428571],  
       [1.        , 0.        , 0.        , 0.        , 0.91304348,  
        0.88571429],  
       [0.        , 0.        , 0.        , 1.        , 1.        ,  
        1.        ],  
       [1.        , 0.        , 0.        , 0.        , 0.43478261,  
        0.54285714]])
```